



TRƯỜNG ĐẠI HỌC BÁCH KHOA - ĐHQG TP.HCM

Khoa Khoa học và Kỹ thuật máy tính | Thực tập 2

PHÂN TÍCH HIỆU NĂNG CỦA JAVASCRIPT VÀ WEBASSEMBLY TRONG XỬ LÝ VIDEO THỜI GIAN THỰC TRÊN TRÌNH DUYỆT WEB

Sinh viên: Võ Hoàng Quốc - 2470310

GVHD: PGS.TS Thoại Nam

TP. HCM, 05/2026

Tổng quan

- Chương 1: Giới thiệu
- Chương 2: Cơ sở lý thuyết
- Chương 3: Thiết kế và cài đặt hệ thống
- Chương 4: Kết quả và phân tích
- Chương 5: Kết luận và hướng phát triển

Chương 1: Giới thiệu

- Bối cảnh đề tài
- Các vấn đề nghiên cứu
- Các nghiên cứu liên quan
- Khoảng trống nghiên cứu
- Mục tiêu nghiên cứu
- Câu hỏi nghiên cứu

Chương 1: Giới thiệu

Bối cảnh đề tài

- Xu hướng client-side computing giúp giảm độ trễ, tiết kiệm băng thông và bảo vệ dữ liệu camera/video.
- Video 4K UHD tạo áp lực rất lớn: 8.3 triệu pixel và khoảng 33 MB dữ liệu thô mỗi khung hình.
- Với mục tiêu 60 FPS, ngân sách xử lý chỉ còn 16.67 ms cho mỗi frame.

FRAME SIZE

33MB

RGBA 4K/frame

60 FPS BUDGET

16.67 ms

per frame

PIXELS

8.3M

3840 x 2160

PIPELINE

real-time

video/media workload

Chương 1: Giới thiệu

Vấn đề nghiên cứu

JavaScript JIT

Kiểu động, warm-up và deoptimization làm thời gian xử lý khó dự đoán.

Garbage Collection

Cấp phát liên tục cho frame 4K tạo latency spikes và jitter.

Event loop đơn luồng

Khó khai thác đa nhân CPU và khó giữ main thread ổn định.

SIMD khó kiểm soát

JS phụ thuộc auto-vectorization của engine, không ổn định với workload pixel.

Bridge và bộ nhớ

Đưa tính toán sang Wasm vẫn đòi hỏi copy-in/copy-out nếu pipeline chưa zero-copy.

Câu hỏi trung tâm:

Wasm có thể đưa lõi xử lý video 4K về dưới ngân sách thời gian thực hay không?

Chương 1: Giới thiệu

Các nghiên cứu liên quan

Wasm vs JS/native

Haas et al.; Jangda et al.; Yan et al. - Wasm có lợi thế, nhưng hiệu năng phụ thuộc runtime và overhead.

Computer vision trên Web

Taheri et al. với OpenCV.js chứng minh tính khả thi của thị giác máy tính trên trình duyệt.

SIMD và convolution

Oishi et al. cho thấy Wasm SIMD tăng tốc các bộ lọc ảnh, đặc biệt với dữ liệu pixel lớn.

Frontend high-performance

Parab tổng hợp vai trò mở rộng của Wasm trong ứng dụng web hiện đại.

Kết luận rút ra

- Đã có nhiều bằng chứng về lợi thế Wasm trong tác vụ bị giới hạn bởi CPU.
- Ít công trình đánh giá video 4K thời gian thực trên pipeline cụ thể.
- Các chỉ số P99/Jitter thường ít được xem xét (so với giá trị trung bình).

Chương 1: Giới thiệu

Khoảng trống nghiên cứu

1. Workload cực lớn

Thiếu đánh giá trên pipeline video 4K UHD, nơi CPU và băng thông bộ nhớ đều bị đẩy lên cao.

2. Tối ưu tích lũy

Nhiều nghiên cứu chỉ hỗ trợ so sánh riêng lẻ JS/Wasm hoặc SIMD/Scalar, ít phân tích từ Scalar → SIMD → MT → Hybrid

3. Độ ổn định

Mean latency chưa đủ cho video thời gian thực; cần nhìn P99, độ lệch chuẩn và jitter.

→ Đề tài lấp đầy khoảng trống bằng benchmark có kiểm soát trên motion detection 4K.

Chương 1: Giới thiệu

Mục tiêu nghiên cứu

Benchmark

hệ thống đo lường chuẩn
hóa

So sánh

JavaScript vs Wasm

Đánh giá

4 mức tối ưu Wasm

Khuyến nghị

trade-off kỹ thuật

- Cô lập chi phí thực thi lỗi khởi decode/render khi đo latency thuật toán.
- Ghi nhận cả tốc độ và tính ổn định: latency, FPS, P99, sigma, speedup.
- Xác định bottleneck khi chuyển từ compute-bound sang memory/bridge-bound.

Chương 1: Giới thiệu

Câu hỏi nghiên cứu

RQ1 - Wasm Scalar

Wasm Scalar tăng tốc như thế nào so với JavaScript trong xử lý video 4K?

RQ2 - SIMD

SIMD 128-bit giảm latency các phép xử lý pixel ở mức nào?

RQ3 - Multi-threading

Đa luồng có đạt tăng trưởng tuyến tính hay bị giới hạn bởi overhead và băng thông bộ nhớ?

RQ4 - Hybrid stability

Kết hợp SIMD + Multi-thread có giữ được P99/Jitter ổn định trong pipeline dài hạn?

Chương 2: Cơ sở lý thuyết

- Các định nghĩa
- Thuật toán phát hiện chuyển động

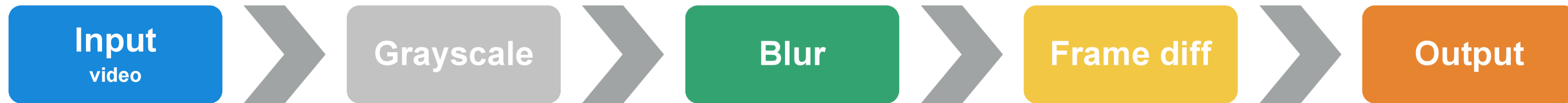
Chương 2: Cơ sở lý thuyết

Các định nghĩa

- **WebAssembly:** Mã nhị phân kiểu tĩnh, AOT/streaming compilation, hiệu năng gần native.
- **Linear Memory:** Mảng byte liên tục, quản lý thủ công, tái sử dụng buffer để giảm GC.
- **SIMD 128-bit:** Xử lý nhiều pixel trên một lệnh, phù hợp grayscale và frame differencing.
- **Web Workers/pthreads:** Khai thác nhiều lõi CPU bằng SharedArrayBuffer và mô hình fork-join.
- **Motion Detection:** Frame differencing sau grayscale và blur, workload lặp lại trên từng pixel.
- **Các chỉ số đánh giá:**
 - Mean/Median phản ánh tốc độ diễn hình
 - P99 và sigma phản ánh jitter

Chương 2: Cơ sở lý thuyết

Thuật toán phát hiện chuyển động



Lý do chọn thuật toán phát hiện chuyển động:

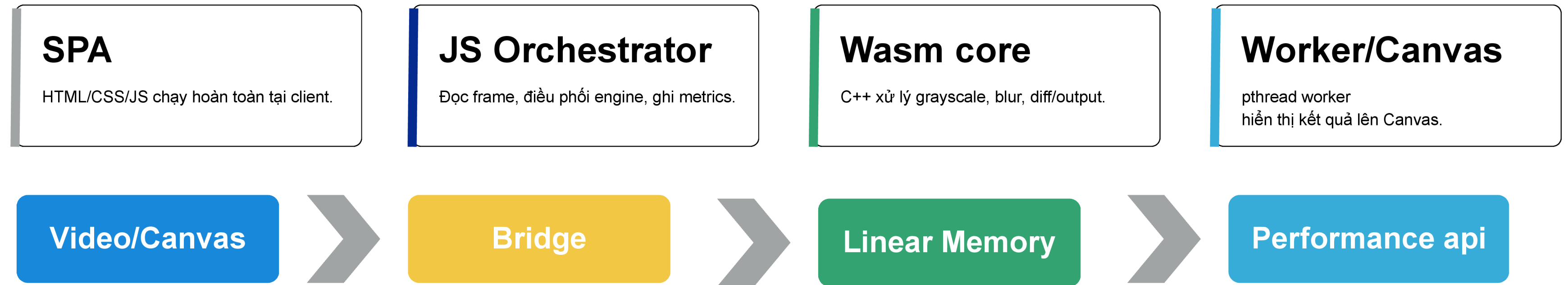
- Thuật toán có tính song song cao, phù hợp để đánh giá SIMD và đa luồng.
- Blur là nút thắt chính của pipeline
- Tối ưu separable filter + sliding window giảm độ phức tạp xử lý.

Chương 3: Thiết kế và cài đặt hệ thống

- Kiến trúc hệ thống benchmark
- Các kịch bản thực nghiệm
- Môi trường và các chỉ số đánh giá

Chương 3: Thiết kế và cài đặt hệ thống

Kiến trúc hệ thống benchmark



Nguyên tắc: giữ cùng video, cùng thuật toán logic, chỉ thay đổi kỹ thuật tối ưu ở tầng Wasm.

Chương 3: Thiết kế và cài đặt hệ thống

Các kịch bản thực nghiệm

JS baseline

TypedArray + JIT + GC

Wasm Scalar

C++ scalar + linear memory

Wasm SIMD

SIMD 128-bit + integer math

Wasm MT

pthread + SharedArrayBuffer

Hybrid

SIMD + Multi-threaded +
optimized blur

Chuỗi kịch bản cho phép đo đóng góp riêng lẻ và tích lũy của từng tầng tối ưu.

Chương 3: Thiết kế và cài đặt hệ thống

Môi trường và các chỉ số đánh giá

Dữ liệu

Video 4K UHD, 3840 x 2160, khoảng 33 MB/frame.

Runtime

Google Chrome/V8, WebAssembly SIMD, SharedArrayBuffer.

Phần cứng

Apple M4, 16GB Unified Memory; GPU không tham gia tính toán lỗi.

LATENCY

ms/frame

thời gian xử lý lỗi

FPS

frames/s

thông lượng đầu cuối

P99

tail

độ trễ xấu

Độ lệch chuẩn

σ

Đo lường độ biến động

Speedup

x lần

So độ cải thiện với baseline

Chương 4: Kết quả và Phân tích

- Kịch bản 1: JavaScript vs WebAssembly Scalar
- Kịch bản 2: JavaScript vs WebAssembly SIMD
- Kịch bản 3: JavaScript vs WebAssembly MT
- Kịch bản 4: JavaScript vs WebAssembly Hybrid (SIMD + MT)

Chương 4: Kết quả và Phân tích

Kịch bản 1: JavaScript vs WebAssembly Scalar

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	35	126	
FPS trung bình	3.4	11.4	+235.3%
Mean (ms)	273.74ms	61.22ms	-77.6%
Median (ms)	258.70ms	60.70ms	-76.5%
P99	462.80ms	64.10ms	-86.1%
Độ lệch chuẩn (σ)	40.82ms	2.89ms	-92.9%

Nhận xét:

- WebAssembly Scalar đã cải thiện rõ rệt cả thông lượng, độ trễ và độ ổn định so với JavaScript.
- Kết quả cho thấy Wasm là nền tảng phù hợp hơn cho các pipeline xử lý video 4K thời gian thực, ngay cả khi chưa sử dụng SIMD hoặc đa luồng.

Chương 4: Kết quả và Phân tích

Kịch bản 2: JavaScript vs WebAssembly SIMD

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	35	218	
FPS trung bình	3.3	20.7	+527.3%
Mean (ms)	284.47ms	23.72ms	-91.7%
Median (ms)	271.30ms	23.40ms	-91.4%
P99	454.50ms	25.90ms	-94.3%
Độ lệch chuẩn (σ)	42.20ms	1.44ms	-96.6%

Nhận xét:

- Kịch bản SIMD mang lại bước nhảy lớn về thông lượng và độ ổn định.
- Chú ý: hiệu năng cải thiện là tác động tổng hợp của SIMD và tối ưu thuật toán blur, không phải chỉ riêng SIMD.

Chương 4: Kết quả và Phân tích

Kịch bản 3: JavaScript vs WebAssembly MT

Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	36	184	
FPS trung bình	3.4	17.3	+408.8%
Mean (ms)	273.9ms	33.75ms	-87.7%
Median (ms)	262.65ms	32.95ms	-87.5%
P99	487.25ms	39.71ms	-91.9%
Độ lệch chuẩn (σ)	44.26ms	7.75ms	-82.5%

Nhận xét:

- WebAssembly đã luồng cải thiện mạnh thông lượng so với JavaScript, nhưng đánh đổi bằng độ ổn định thấp hơn Wasm Scalar.
- Nguyên nhân chính đến từ chi phí scheduling, đồng bộ hóa Fork-Join và tranh chấp tài nguyên bộ nhớ khi xử lý dữ liệu 4K.

Chương 4: Kết quả và Phân tích

Kịch bản 4: JavaScript vs WebAssembly Hybrid (SIMD + MT)

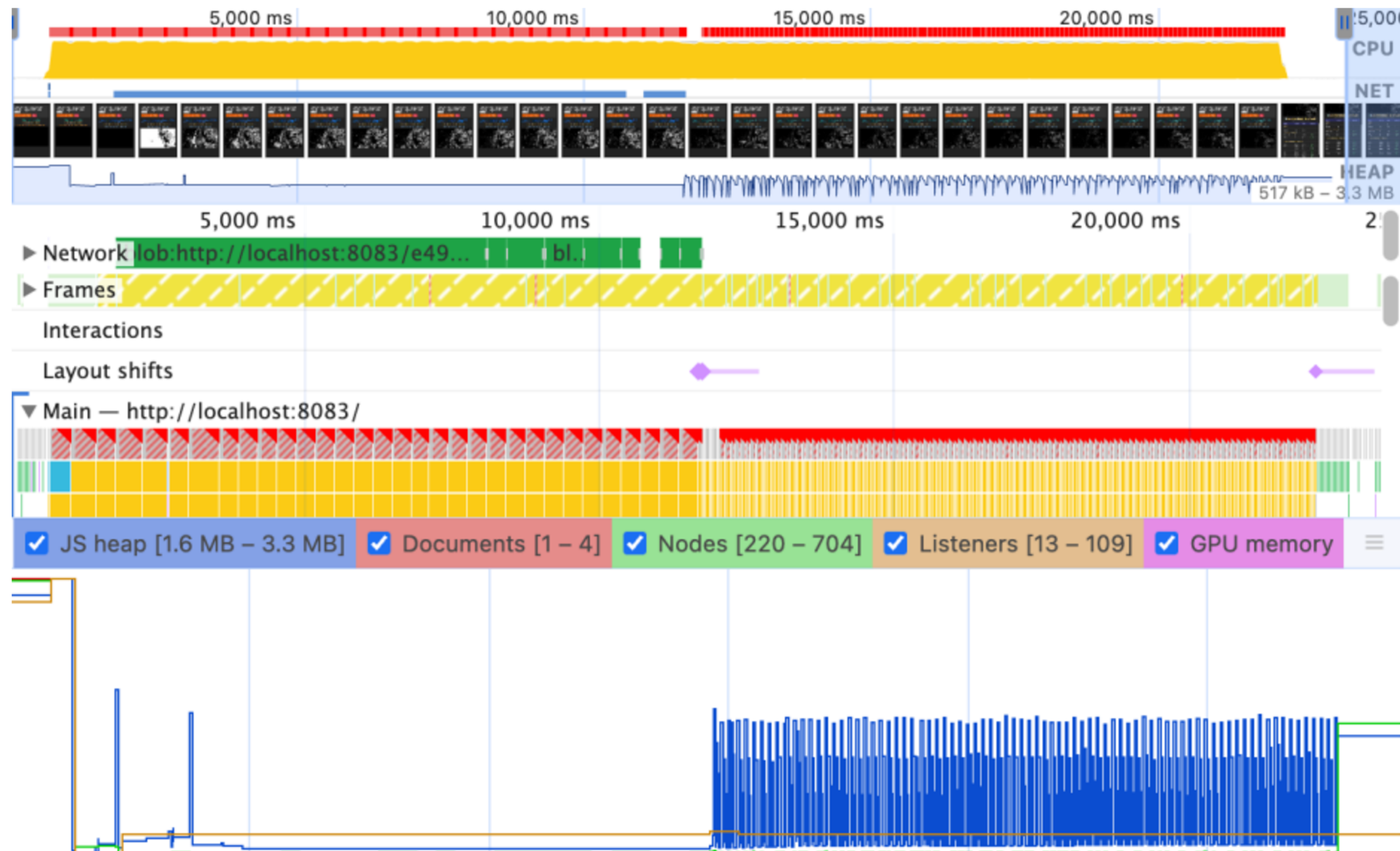
Chỉ số	JavaScript	WebAssembly	Sự khác biệt
Tổng khung hình	36	290	
FPS trung bình	3.4	27.9	+720.6%
Mean (ms)	272.24ms	11.13ms	-95.9%
Median (ms)	256.05ms	11.12ms	-95.7%
P99	440.37ms	13.17ms	-97%
Độ lệch chuẩn (σ)	38.96ms	1.29ms	-96.7%

Nhận xét:

- Kịch bản Hybrid là cấu hình hiệu quả nhất, đạt mức tăng tốc khoảng 8.21 lần so với JavaScript.
- Dù lõi thuật toán đã đáp ứng ngân sách 60 FPS, hiệu năng đầu cuối vẫn bị giới hạn bởi chi phí bridge và hiển thị, cho thấy nút thắt cổ chai đã chuyển từ tính toán sang truyền dữ liệu và rendering.

Chương 4: Kết quả và Phân tích

Kịch bản 4: JavaScript vs WebAssembly Hybrid (SIMD + MT)



Nhận xét:

- Nút thắt hiệu năng ở giai đoạn này nằm ở JavaScript–Wasm bridge và pipeline hiển thị.
- Để tiếp tục cải thiện FPS, cần giảm sao chép bộ nhớ, tái sử dụng buffer và tối ưu đường truyền dữ liệu từ Wasm sang Canvas.

Chương 4: Kết luận và Hướng phát triển

- Kết luận và đóng góp
- Hạn chế và Hướng phát triển

Chương 5: Kết luận và Hướng phát triển

Kết luận và đóng góp

1. Benchmark 4K

Xây dựng framework đo lường video 4K trên trình duyệt, có tách chỉ số latency lỗi và FPS đầu cuối.

2. Bảng chứng định lượng

Wasm Scalar nhanh hơn JS 4.47x; Hybrid đưa core latency xuống 11.13 ms/frame.

3. P99/Jitter

Chứng minh mean latency/FPS chưa đủ; P99 và sigma phản ánh chất lượng thời gian thực.

4. SIMD then chốt

Vector hóa pixel mang lại lợi ích rõ hơn đa luồng đơn lẻ trong workload này.

5. Bottleneck mới

Khi lỗi đã nhanh, giới hạn dịch sang memory bandwidth và JS-Wasm bridge.

Chương 5: Kết luận và Hướng phát triển

Hạn chế và Hướng phát triển

Hạn chế

Mới đánh giá trên một cấu hình phần cứng/trình duyệt; chưa đo năng lượng, nhiệt độ và ảnh hưởng của tác vụ web song song.

Bottleneck còn lại

End-to-end bị giới hạn bởi copy-in/copy-out, tạo ImageData, Canvas rendering và memory churn.

Hướng kỹ thuật

Kết hợp Wasm, SIMD, đa luồng với WebCodecs, WebGPU và thiết kế zero-copy/shared-memory pipeline.

Đề án tốt nghiệp

Phát triển ứng dụng phát hiện chuyển động video/camera thời gian thực trên Web.

Tài liệu tham khảo

- [1] Jangda, A., Powers, B., Berger, E.D. and Guha, A. (2019) 'Not So Fast: Analyzing the Performance of WebAssembly vs. Native Code', Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19), pp. 107–120. Available at: <https://www.usenix.org/conference/atc19/presentation/jangda>
- [2] Yan, Y., Tu, T., Zhao, L., Zhou, Y. and Wang, W. (2021) 'Understanding the Performance of WebAssembly Applications', Proceedings of the 21st ACM Internet Measurement Conference (IMC '21), pp. 533–549. doi: <https://dl.acm.org/doi/10.1145/3487552.3487827>
- [3] Taheri, S., Veidenbaum, A., Nicolau, A., Hu, N. and Haghighat, M.R. (2018) 'OpenCV.js: Computer Vision Processing for the Open Web', Proceedings of the 9th ACM Multimedia Systems Conference (MMSys '18), pp. 478–483. doi: <https://doi.org/10.1145/3204949.3208126>
- [4] Țălu, M. (2025) 'A Comparative Study of WebAssembly Runtimes: Performance Metrics, Integration Challenges, Application Domains, and Security Features', Archives of Advanced Engineering Science, pp. 1–13. doi: <https://doi.org/10.47852/bonviewAAES52024965>
- [5] Haas, A., Rossberg, A., Schuff, D.L., Titzer, B.L., Holman, M., Gohman, D., Wagner, L., Zakai, A. and Bastien, J.F. (2017) 'Bringing the Web up to Speed with WebAssembly', Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '17), pp. 185–200. doi: <https://doi.org/10.1145/3140587.3062363>
- [6] Ciszewski, J. and Myk, K. (2025) 'Performance Comparison of Web Assembly and JavaScript', Journal of Pioneering Artificial Intelligence Research, 1(2), pp. 1–7. doi: <https://doi.org/10.63721/25JPAIR0111>
- [7] Oishi, S., Ishikawa, K., Nogami, H. and Fukushima, N. (2023) 'Performance Evaluation of Image Convolution with WebAssembly', Proceedings of SPIE: International Workshop on Advanced Imaging Technology (IWAIT) 2023, 12592, 125922G. doi: <https://doi.org/10.1117/12.2667004>
- [8] Vemuri, V.P.K. (2021) 'WebAssembly for High-Performance Web Applications: A Study on Execution Speed and Efficiency', International Journal on Science and Technology, 12(2). doi: <https://doi.org/10.71097/IJSAT.v12.i2.2812>
- [9] Parab, A. (2025) 'WebAssembly's Expanding Role in Frontend Development: Transforming Web Application Capabilities', International Journal of Computational and Experimental Science and Engineering, 11(4), pp. 8694–8702. doi: <https://doi.org/10.22399/ijcesen.4281>

***Cảm ơn mọi người
đã lắng nghe!!!***

